CIS*3110 Operating Systems

Week 9
Lecture #13 Threads intro

Prof. Yulia Kotseruba
March 3, 2026

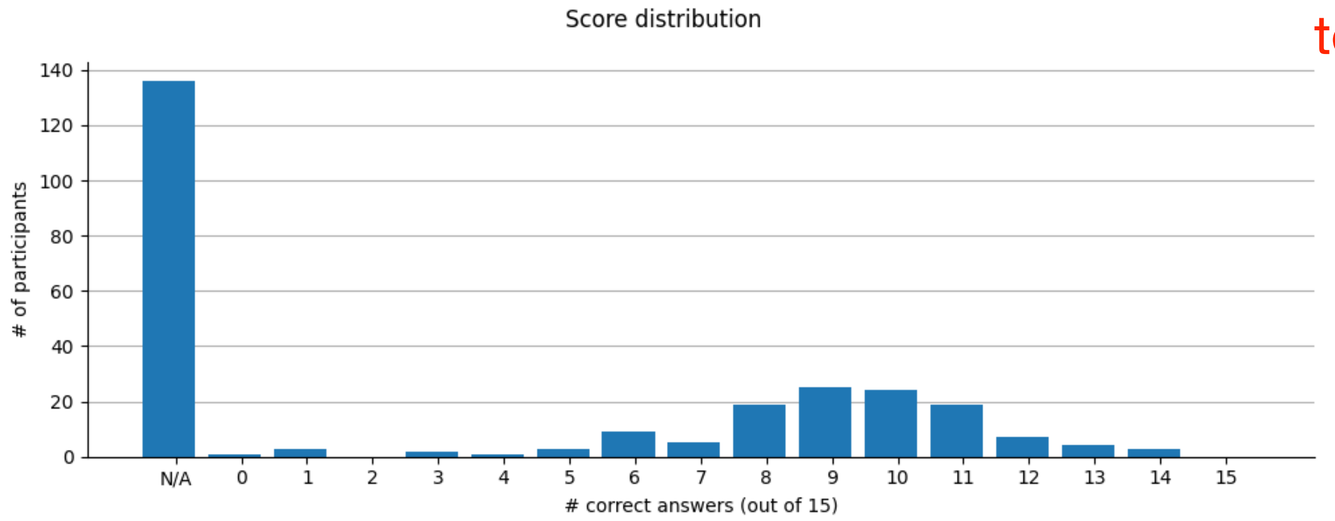
Quiz 11

```
int main(){
    int* arr = malloc(10);
    arr = malloc(20);
    free(arr);
    arr = malloc(40);
    return 0;
}
```

this is the only memory freed

- **59%** average (up from 56% in quiz 10)
- 3 hardest questions:
  - (32%) What is /proc/pid/maps in Linux? a) a text file b) a binary file c) a directory d) a process
  - (38%) How much memory is directly lost when this code executes? a) 10 b) 20 c) 40 d) 50
  - (38%) Starvation in RR happens when ____ a) all jobs arrive together b) when q is small c) when q is large d) never happens

10 and 40 bytes are lost because return from main terminates the process



Bonus: exit vs return

```
void func() {
  printf("func() runs\n");
  return;
}

int main() {
  func();
  printf("after func()\n");
}
```

ret_test.c

return - returns from the function

```
void func() {
  printf("func() runs\n");
  exit(0);
}

int main() {
  func();
  printf("after func()\n");
}
```

exit_test.c

exit - terminates the calling process from any point in the code, flushes buffers, etc

this line is never reached because exit() terminated the process at the end of func() call

```
>>> ./ret_test
func() runs
after func()
```

```
>>> ./exit_test
func() runs
```

return and exit behave the same only at the end of main() -> return from main calls exit behind the scenes

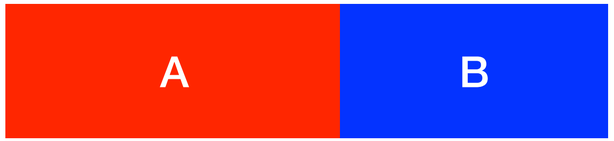
Recap: Virtualization

- We finished **virtualization** of CPU and memory
- Takeaways
  - Process API: `fork(), wait(), exec(), kill()`
  - Scheduling algos: non-preemptive (FIFO, SJF), preemptive (PSJF, RR, MLFQ, Stride, Lottery, CFS, EEVDF)
  - Process API + CPU scheduling -> can run more processes than there are CPU cores
  - Virtual memory + swapping -> more memory than there is RAM available
  - Memory allocation and address translation -> segmentation, paging, paging+segmentation, inverted page table
  - Cache replacement (TLB and RAM) -> MIN, RAND, FIFO, LRU, clock

Concurrency

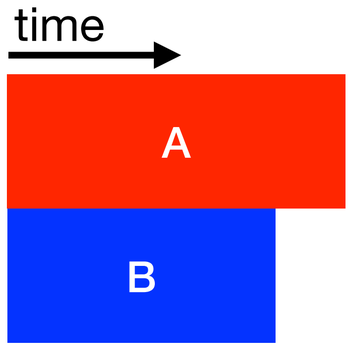
- Working on the tasks simultaneously
- Not the same as parallelism

Serial execution
(no concurrency)



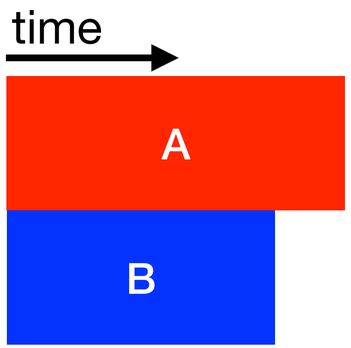
Serial means that the processes run one after another, there is no overlap in time between them

Parallell
execution

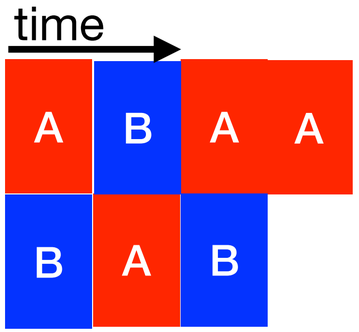


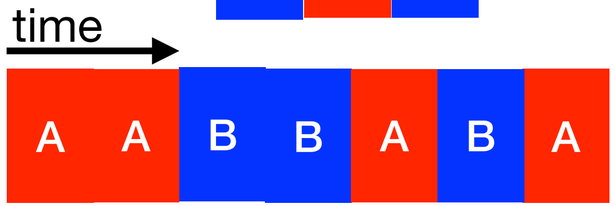
Parallel means that processes run at the same time

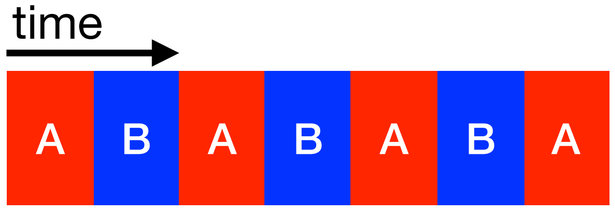
Concurrent
execution



Concurrency is broader, means tasks are worked on within the same time window but not necessarily in parallel







Concurrency

- Goal: make computer **faster**
- Option #1: buy/build a faster computer
- Option #2: use hardware more efficiently by performing **many** tasks **simultaneously**
  - processes
  - threads aka lightweight processes

Process

- Process is a running program with
  - code
  - data
  - heap
  - stack
  - registers
  - program counter
  - state
  - I/O stuff
  - etc.
  - process ID

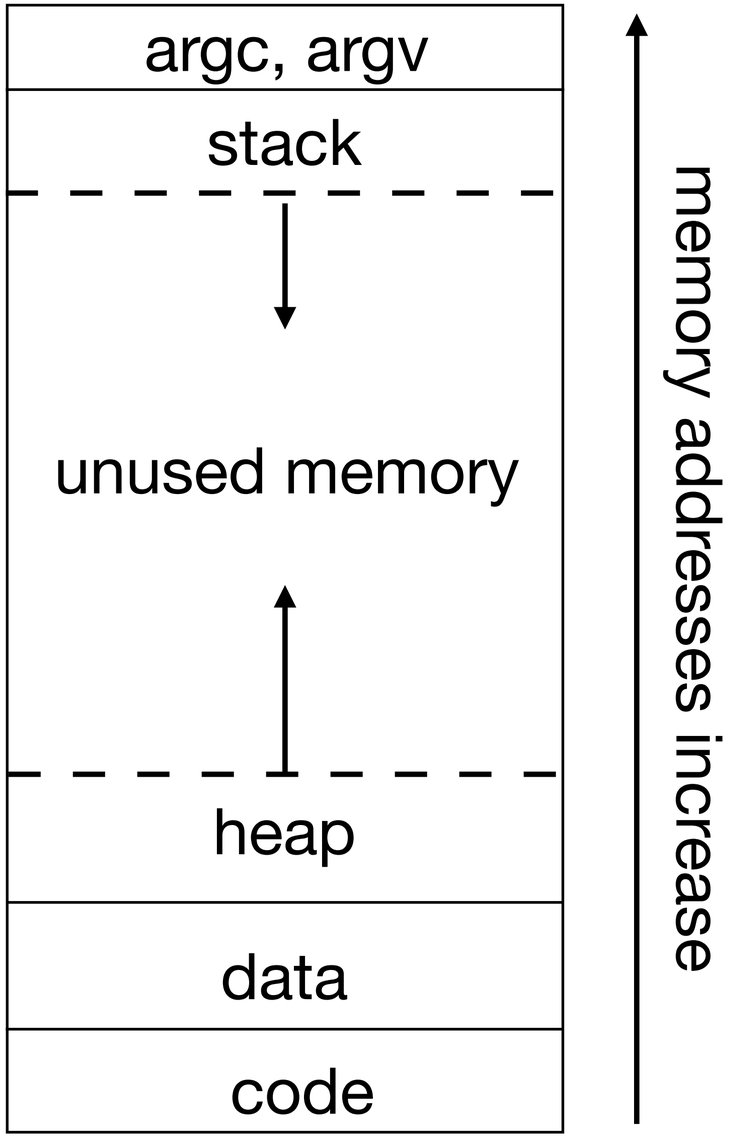
Processes are **independent** running programs that have their **own** virtual memory

e.g. my laptop runs Keynote, Mail, Teams, etc., which do not share anything with each other (hopefully)

Process

- Process is a running program with
  - code
  - data
  - heap
  - stack
  - registers
  - program counter
  - state
  - I/O stuff
  - etc.
  - process ID

single-threaded process



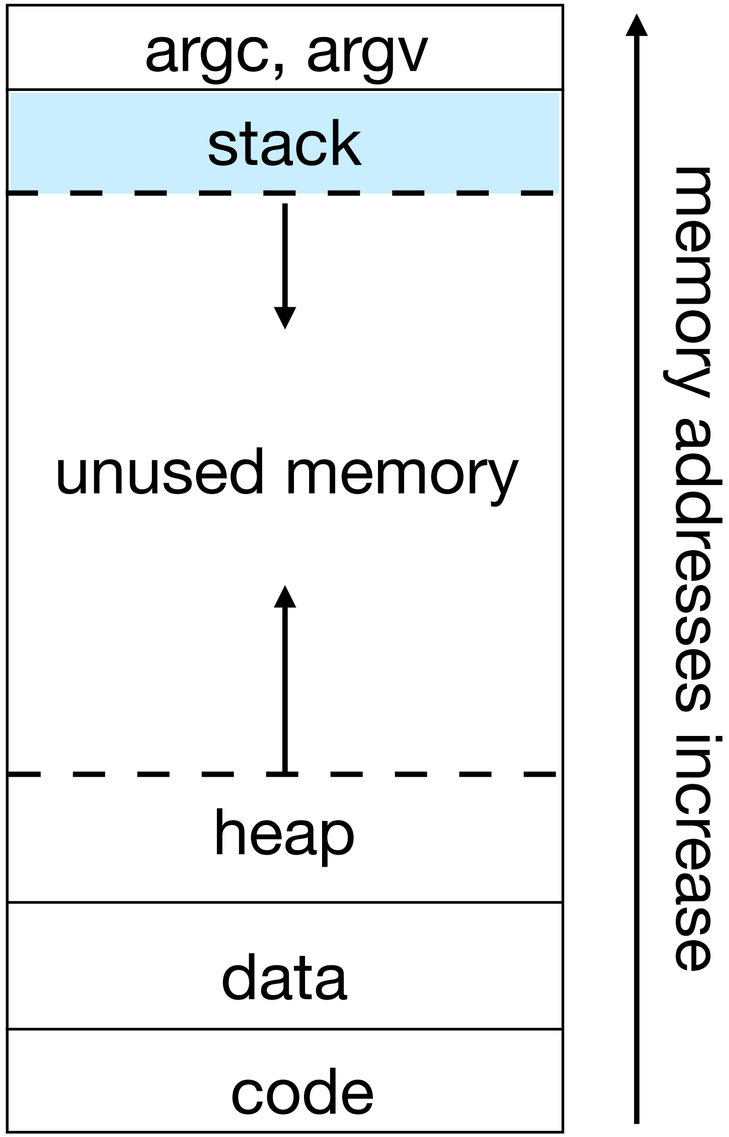
Process

- Process is a running program with
  - code
  - data
  - heap
  - stack
  - registers
  - program counter
  - state
  - I/O stuff
  - etc.
  - process ID
  - thread ID

and **at least 1 thread** which has registers, stack, and program counter

each thread is an independent serial execution

single-threaded process



Process

- Process is a running program with
  - code
  - data
  - heap
  - stack
  - registers
  - program counter
  - state
  - I/O stuff
  - etc.
  - process ID
  - thread ID
  - thread ID

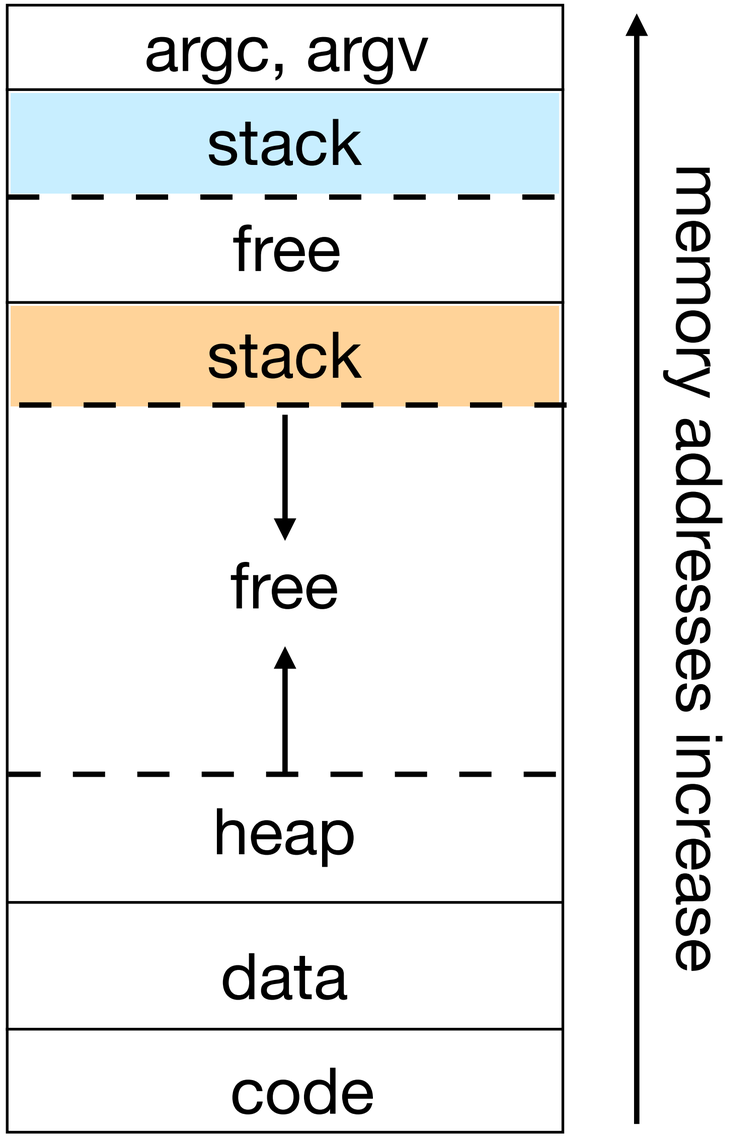
there can be **> 1 thread** in a process

- stack
- registers
- program counter

Threads share:
code and global variables

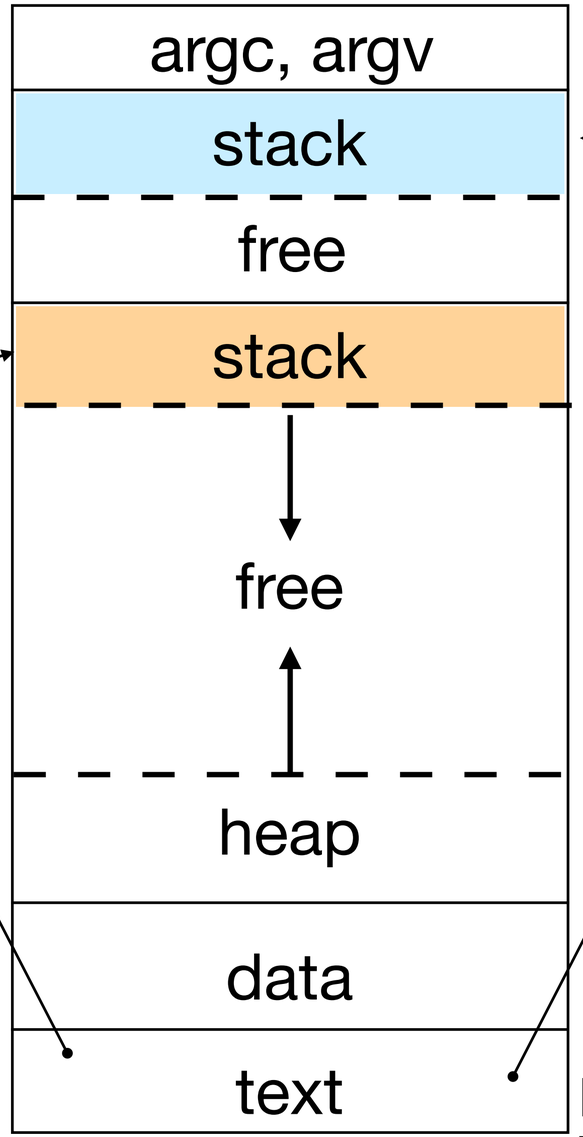
Threads **do not** share:
instruction pointer, registers, stack, id

multi-threaded process



Threads

multi-threaded process



each thread has its own stack where it saves results of its computation, function arguments, etc

Thread 1

Thread 2

Each thread will have its own

- id
- program counter (current instruction being executed)
- registers

everything else in the address space is shared

program counters of threads can be different

Threads

- Each thread has a Thread Control Block (TCB)
  - link to PCB
  - thread id
  - thread state
  - thread priority
  - stack pointer
  - registers
  - and more

Thread API

- pthread (POSIX* threads) is a default UNIX standard
  - `pthread_create()` - create a thread
  - `pthread_join()` - wait for a thread to terminate, detaches the thread and returns its exit status
  - `pthread_detach()` - does not terminate the thread, reclaim resources used by the thread after it ends
  - `pthread_exit()` - terminates calling thread, other waiting threads can see its exit status
- most pthreads functions return 0 on success and `errno` on failure

*Portable Operating System Interface - a standard defining the interface

Creating threads

- `pthread_create(thread, attr, start_routine, arg)`
  - creates a new thread
  - `thread` - will contain new thread ID
  - `attr` - thread attributes or NULL for defaults
  - `start_routine` - a function that the thread will execute
  - `arg` - a single argument for `start_routine`

```
>>> ./pthread_create
[thread] 23615860801 83
[thread] 23531319361 83
[proc] pid=83 t1=23615860801 t2=23531319361
```

```c
//includes omitted
void *print_ids(void*) {
    pthread_t tid = pthread_self();
    printf("[thread] %ul %d\n", pthread_self(), getpid());
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, print_ids, NULL);
    pthread_create(&t2, NULL, print_ids, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("[proc] pid=%d t1=%ul t2=%ul\n", getpid(), t1, t2);
    return 0;
}
```

pthread_create.c

this function just prints a thread id and id of its parent process

this will create a thread with default attributes, populate t1 with its id, and the thread will run print_ids

this creates an identical second thread

this waits for both threads to terminate

```
>>> ./pthread_create_1
[main] stack=0xffffdb848534, shared=0, heap=99
[thread] stack=0xffffb897e93c, shared=0, heap=99
```

main and thread have their own stack and stack variables but share the heap

```
int shared = 0;

void *print_thr(void *arg) {
    int local = 42;
    printf("[thread] stack=%p shared=%d heap=%d\n", &local, shared, *(int*)arg);
    return NULL;
}

int main(void) {
    int local = 1;
    int *heap = malloc(sizeof *heap);
    *heap = 99;
    printf("[main] stack=%p shared=%d heap=%d\n", &local, shared, *heap);

    pthread_t t;
    pthread_create(&t, NULL, print_thr, heap);
    pthread_join(t, NULL);
    free(heap);
}
```

thread has its stack var

main process has a local stack variable and allocates a heap variable

pass heap var as an arg to print_thr

create and call thread

pthread_create_1.c

```
>>> ./pthread_create_1
[main] stack=0xffffdb848534, shared=0, heap=99
[thread] stack=0xffffb897e93c, shared=5, heap=99
```

main and thread have their own stack and stack variables but share the heap and data

```
int shared = 0;

void *print_thr(void *arg) {
    int local = 42;
    printf("[thread] stack=%p shared=%d heap=%d\n", &local, shared, *(int*)arg);
    return NULL;
}

int main(void) {
    int local = 1;
    int *heap = malloc(sizeof *heap);
    *heap = 99;
    printf("[main] stack=%p shared=%d heap=%d\n", &local, shared, *heap);
    shared = 5;
    pthread_t t;
    pthread_create(&t, NULL, print_thr, heap);
    pthread_join(t, NULL);
    free(heap);
}
```

if main changes a global variable

thread will print the updated value

pthread_create_1.c

Terminating threads

- `pthread_exit()` - the thread terminates its execution and returns a result early
- `pthread_join()` - usually called by another thread to wait for the thread to terminate and get its return value
  - can be called before or after `pthread_exit()`
- `pthread_detach()` - can be called by either the thread itself or another thread
  - indicates that we do not want to wait for it to finish nor get its return value
  - does not terminate the thread
  - after the thread terminates its resources will be released
  - usually called shortly after creating a new thread

```
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_join_ok.c

this function sleeps for 5 s
and then returns 5

```
>>> ./pthread_join_ok
[thread] slept for 5 s
[main] worker result = 5
```

pthread_join waits for the thread to finish
and gets its return value

print the return value

```
//includes omitted
void *worker(void *arg) {
    pthread_exit(5);
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

this line will return 5 right away

code after exit is unreachable

pthread_join waits for the thread to finish and gets its return value

print the return value

pthread_join_exit.c

```
>>> ./pthread_join_ok
[main] worker result = 5
```

```c
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);
    pthread_join(t, &ret);
    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_join_again.c

resources are freed after the first join

undefined behavior if the same thread is joined twice

```
>>> ./pthread_join_ok
[thread] slept for 5 s
[main] worker result = 5
```

```c
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

what if we do not join the thread

pthread_no_join.c

```
>>> ./pthread_detach_join
[main] worker result = -41815572
```

```c
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_no_join.c

what if we do not join the thread

```
>>> ./pthread_detach_join
[main] worker result = -41815572
```

main will print whatever is stored in the unitialized variable ret and exit, killing the thread

```c
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_detach(t);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_detach.c

what if we detach the thread?

```
>>> ./pthread_detach_join
[main] worker result = -41815572
```

we are not capturing the return from the thread

main will print whatever is in ret

when main exits, its threads are killed

```c
//includes omitted
void *worker(void *arg) {
    sleep(5);
    printf("[thread] slept for 5 s\n");
    return (void*) 5;
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_detach(t);

    pthread_join(t, &ret);

    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_detach_join.c

```
>>> ./pthread_detach_join
[main] worker result = -41815572
```

if we detached the thread, it cannot be joined again

```c
//includes omitted
void *worker(void *arg) {
   int local = 7;
   pthread_exit((void*) &local);
}

int main(void) {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, worker, NULL);

    pthread_join(t, &ret);
    printf("[main] worker result = %d\n", ret);
    return 0;
}
```

pthread_join_local.c

what if we return a pointer to the local variable?

```
>>> ./pthread_join_local
[main] worker result = -1540101828
```

local variable is on thread's stack and will become invalid once the thread exits -> undefined behavior

Summary so far

- Each process has **one or more threads**
- Threads are **lightweight processes**
  - share heap, data, and code
  - have their own stack, program counter, registers, thread id
- pthreads provides a flexible API for managing threads
  - `pthread_create(), pthread_join(), pthread_detach(), pthread_exit()`
- Avoid common pitfalls
  - applying join and detach operations on the same thread
  - joining/detaching the same thread twice
  - returning local thread variables to main

Threads

- Why bother with threads when we already have processes?
  - Reason #1 Better parallelism
  - Reason #2 Reducing overhead

Parallelism

- Task - a unit of computation that can be executed in parallel with other tasks on multiple CPU cores/processors
- Data parallelism
  - performing the **same task** on **different** subsets of **data** in parallel
  - e.g. matrix multiplication
- Task parallelism
  - performing **different tasks** in parallel
  - e.g. one thread does computation and another does I/O

multiplying matrices

A B AxB = C

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

$$c_1 = a_1 * b_1 + a_2 * b_4 + a_3 * b_7$$

each entry in matrix C can be computed independently, i.e. we do not need to know the value of c2 to compute c1

we can compute C in any order and in parallel

Summary

	Process	Thread
Description	Is a program that is executed by one or more threads	A basic unit of CPU computation
Resource sharing	No sharing of heap, data, stack, registers (some processes may share code)	Share heap, code, data Own stack and registers
Address space	Separate	Shared
Communication	Interprocess communication, e.g. pipes/ sockets	Via global variable
Context switching	Slower (need to save/load the entire address space)	Faster (need to update the state)
Parallelism	Each process is scheduled and run independently of other processes	Depends on the multithreading model

Summary

- Pros of threads: shared memory
  - **quicker** to create a thread
    - no need to allocate memory, no cache misses, etc.
  - **faster** context switching
    - do not need kernel involvement,
  - **easier** data sharing
    - exchange data/messages without special mechanisms
  - **parallel** execution
    - can wait for I/O and do useful work simultaneously
- Cons of threads: shared memory
    - **no security** among threads

Reading

- Chapter 26 Concurrency: An introduction
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf
- Chapter 27 Interlude: Thread API
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-api.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-intro
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-api
- Optional reading
  - https://cs.stanford.edu/people/karpathy/advice.html

A2

- LOC results next lecture
- README grades next week

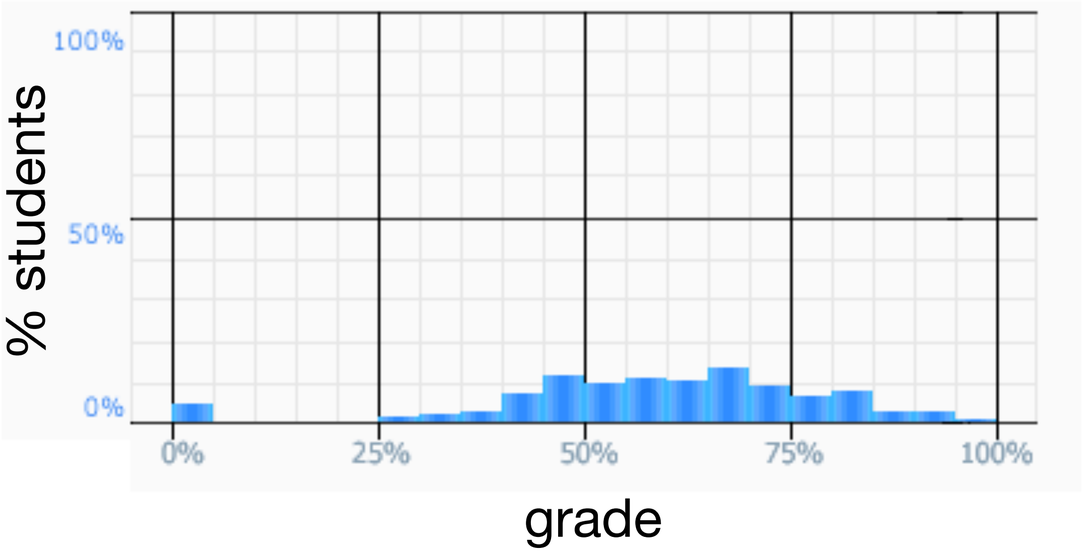
A3

- Due **Mar 16 @ 11:59 PM EST**
  - 12 students started
  - 1 finished
- Do not wait until the weekend to start!

Midterm

- Class average **59%**
- 36 students (**~10%** of the class) received **> 80%**
  - 8 students received > 90%
  - 1 student received 60/60 on MCQ
  - 1 student received 40/40 on short answers
- 60 students (**~23%** of the class) **failed** (<50%)
  - **39** of them **did not attend** most classes
- 12 students did not write the test

- Regrade requests accepted until **March 9**
  - **only through Gradescope**
  - provide a justification for regrade
  - the whole midterm will be regraded

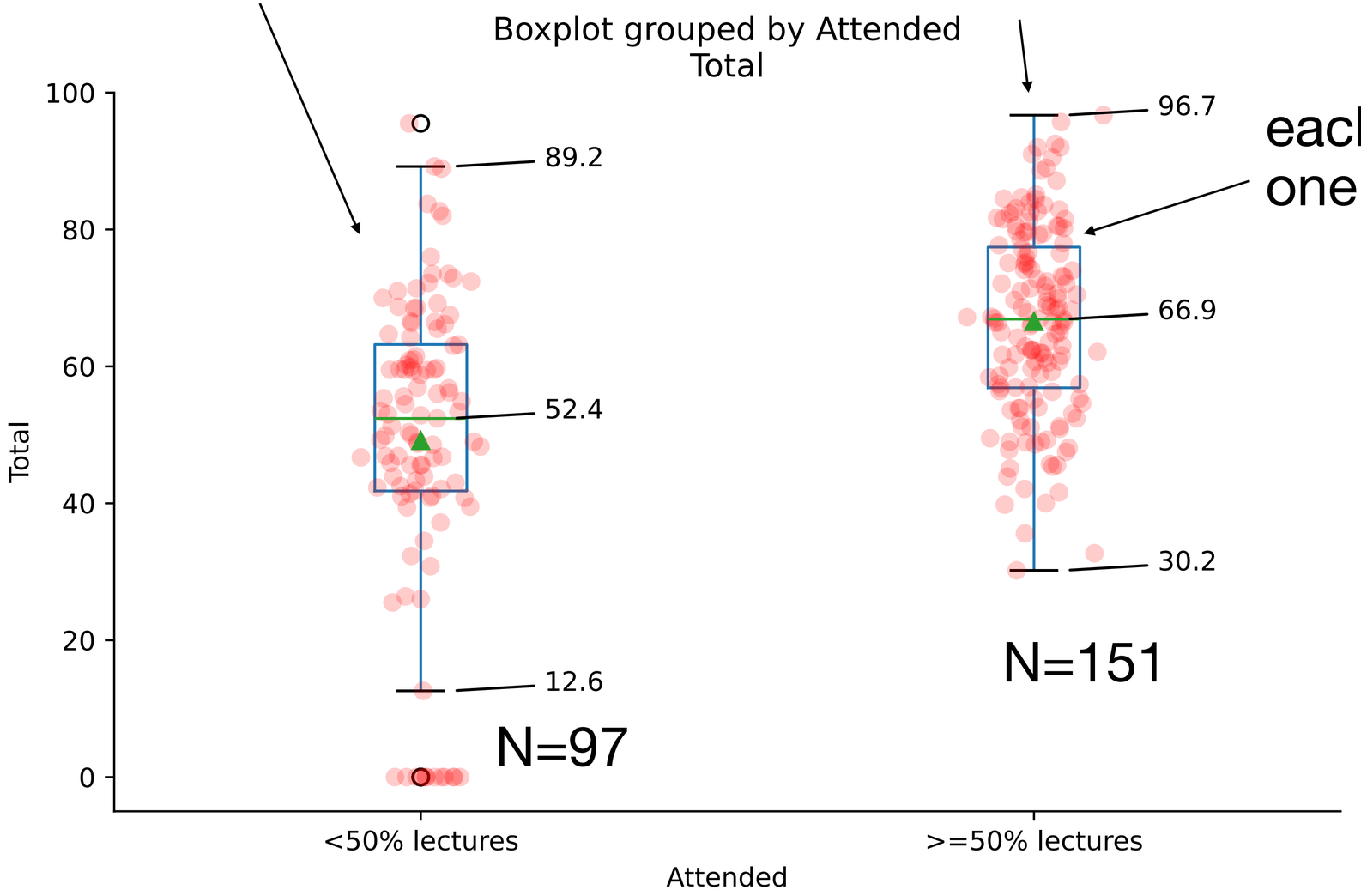


Midterm

I divided grades into 2 groups based on Kahoot quiz results

students who attended < 50% lectures

students who attended >= 50% lectures



each red circle is one grade*

*coordinates are randomized along the horz axis so that the same values do not overlap

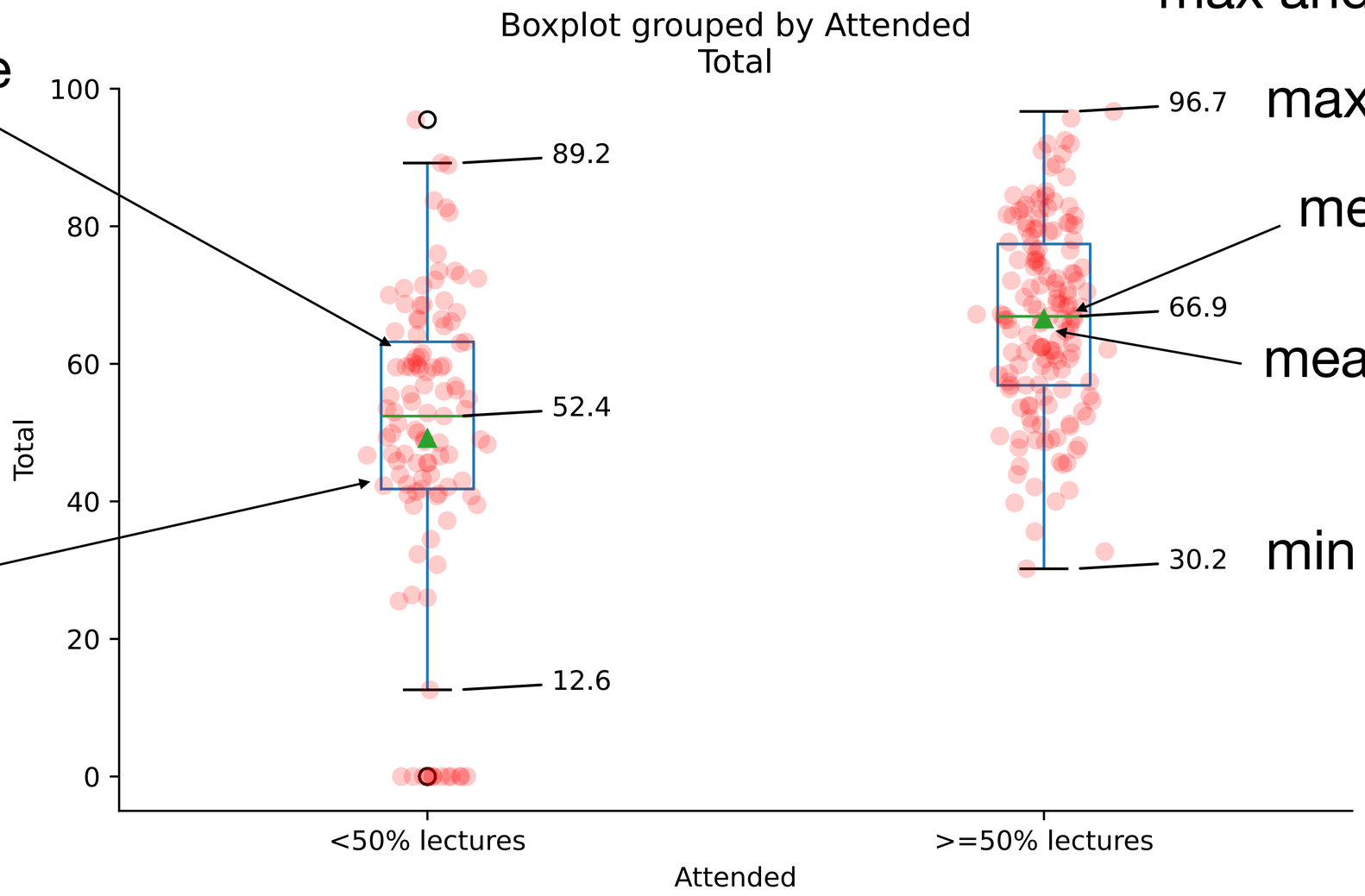
Midterm

each box plot shows the following

whiskers point at the max and min values

top 75% above this line

bottom 25% under this line



max value

median (line)

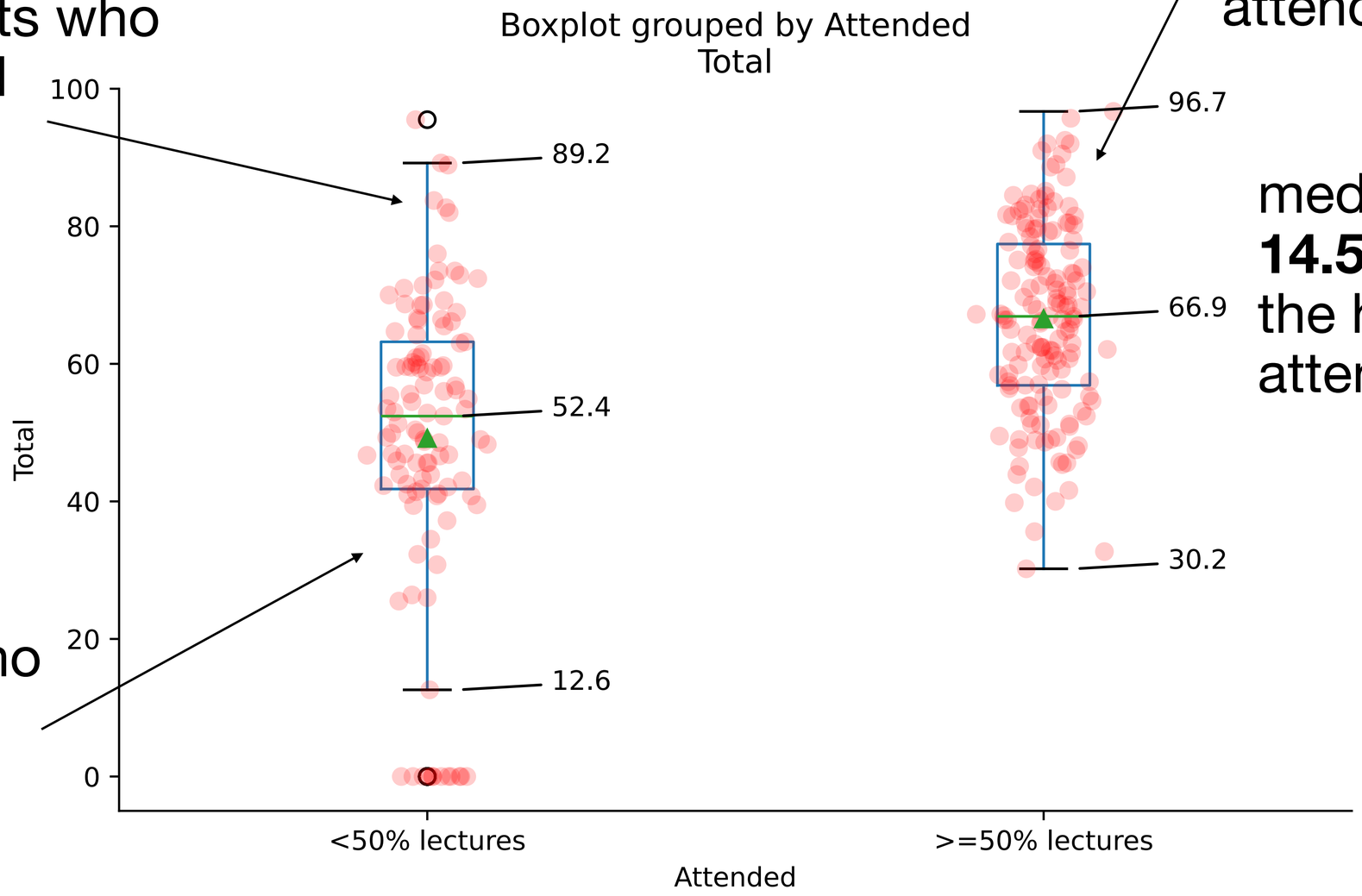
mean (triangle)

min value

Midterm

only **6** students who did not attend achieved 80+

most top marks are in the high attendance group

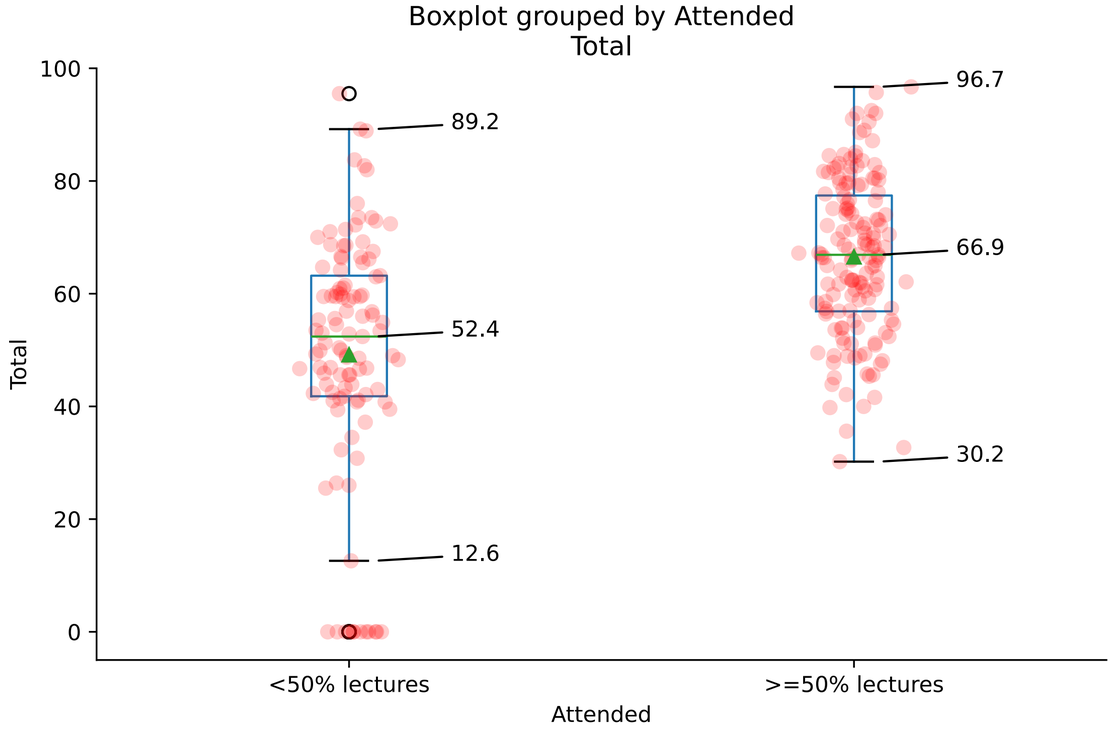


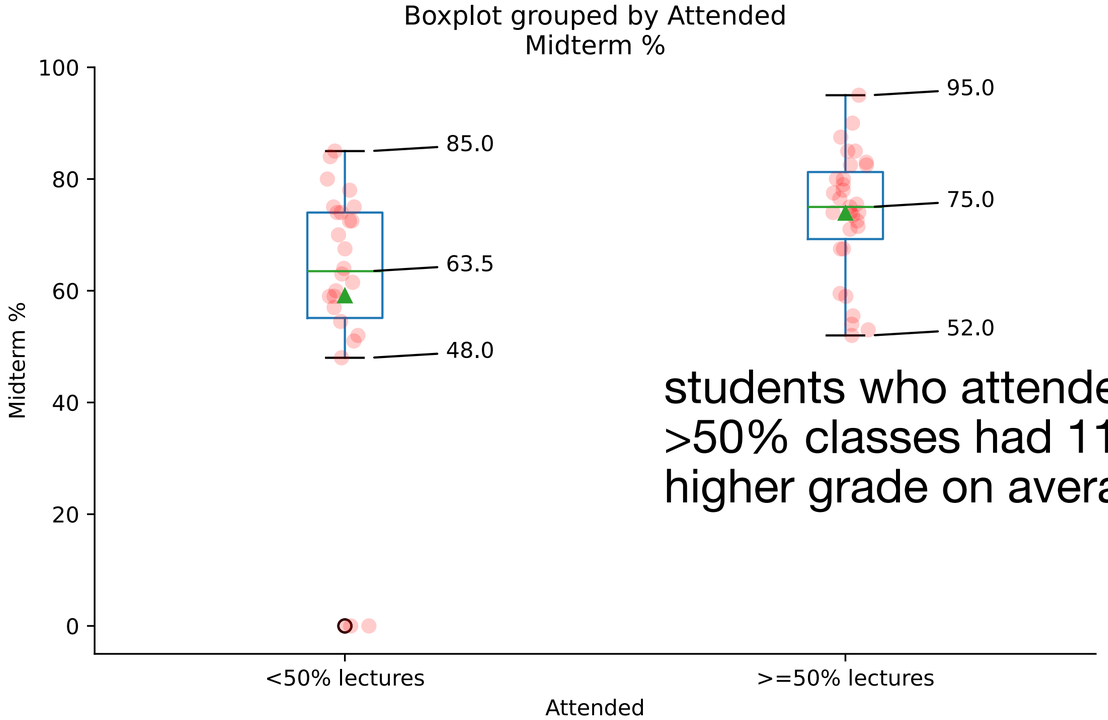
median grade was **14.5% higher** in the high attendance group

39 students who failed did not attend most classes

Midterm

similar trends in
attendance vs grades



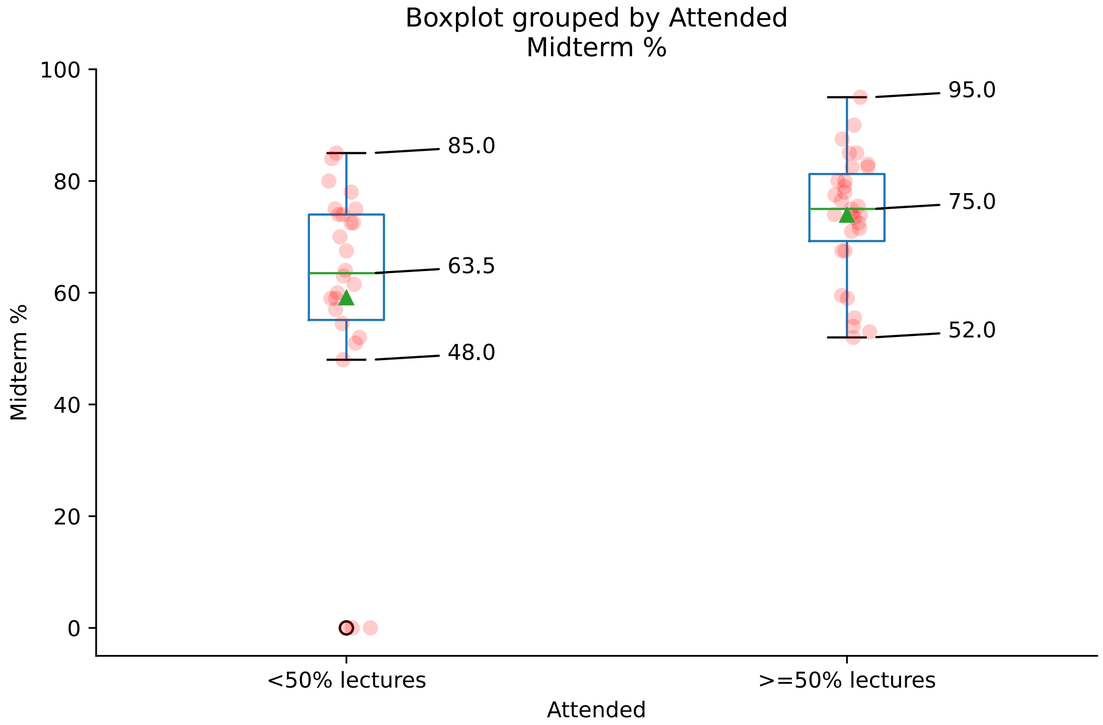


students who attended
>50% classes had 11.5%
higher grade on average

this is from the previous term
(4th year course, 60 students)

Midterm

similar trends in
attendance vs grades

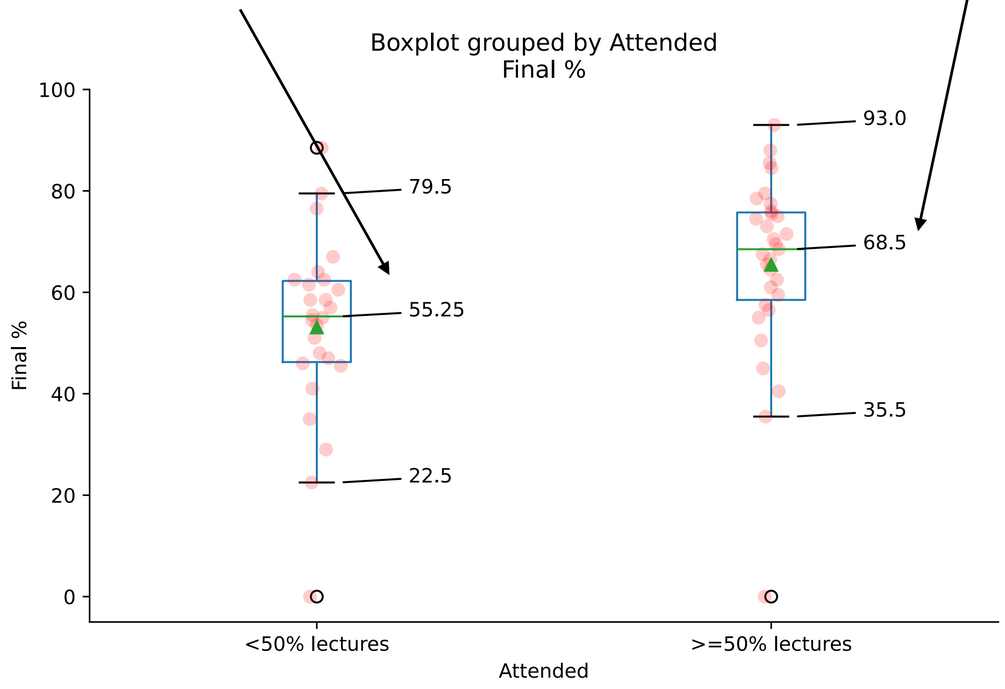


this is from the previous term
(4th year course, 60 students)

median grades dropped

by 8.2% in the low
attendance group

by 6.5% in the
high attendance
group



this is their final

Midterm

- These plots show correlation, not causation
- Attendance **contributes** to good grades but does not guarantee them
  - extra **exposure** to the information
  - class **participation**
  - quiz **practice**
  - better **understanding** of **my** requirements
  - **does not replace studying on your own**

most students who did not do quizzes in class also did not practice them on their own

- Final grades are likely to be **lower** on average
  - I will **not** make the test harder
  - but there will naturally be **more material** covered

Midterm

- These plots show correlation, not causation
- Attendance **contributes** to good grades but does not guarantee them
  - extra **exposure** to the information
  - class **participation**
  - quiz **practice**
  - better **understanding** of **my** requirements
  - **does not replace studying on your own**

most students who did not do quizzes in class also did not practice them on their own

- Final grades are likely to be **lower** on average
  - I will **not** make the test harder
  - but there will naturally be **more material** covered

useful advice for studying/exams

READ THE WHOLE THING

https://cs.stanford.edu/people/karpathy/advice.html

Quiz 12

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999